Actividad 7: Mini Juego "Piedra, Papel o Tijera" con Persistencia de Datos (struct + fstream)

Federico Mattia

Maximo Orue

Ulises Ferrari

Lucas Del Pino

E.E.S.T. N.º 1 “Eduardo Ader”, Vicente López

Curso: Laboratorio de Programación 5° (3° Grupos A-B)

Profesor: Mansilla Muñoz York E.

Fecha: 15/10/2026

Introducción

Este trabajo presenta el desarrollo de la Actividad 7 de Laboratorio de Programación, siguiendo las normas del estilo APA (7.ª edición). La actividad consiste en programar en C++ un mini juego de Piedra, Papel o Tijera contra la inteligencia artificial de la consola, en el que cada partida se guarda en una estructura (*struct*) y se persiste en un archivo de texto (*historial_partidas.txt*) mediante la librería *fstream*.

El propósito central es distinguir la memoria RAM, que es volátil, del almacenamiento secundario, que es no volátil. Para ello se aplican tres ideas: el modelado de datos con *struct*, la escritura en disco con *ofstream* en modo *append* y la lectura desde disco con *ifstream*. El código fuente y la documentación se encuentran en los repositorios del grupo, indicados en el apartado de repositorios.

Desarrollo

Objetivos de la actividad

Los objetivos clave de la actividad fueron los siguientes:

- **Modelado con struct:** agrupar variables heterogéneas (ID, jugador, elección del jugador, elección de la IA y resultado) en una sola entidad.
- **Escritura en disco (ofstream):** guardar los resultados en modo *append* (*ios::app*) para no sobrescribir partidas previas.
- **Lectura desde disco (ifstream):** recuperar el historial completo del archivo .txt y mostrarlo en pantalla.

Entorno de trabajo y herramientas

El programa se escribió en el lenguaje C++ en el archivo *src/main.cpp*, utilizando Visual Studio Code como entorno de desarrollo. La compilación se realizó desde la terminal de PowerShell con el compilador g++ habilitando el estándar C++11, y el control de versiones se gestionó con Git, sincronizando el código en dos repositorios remotos (GitHub y GitLab). Los comandos utilizados fueron:

```
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe
.\src\piedrapapelotijera.exe
```

Repositorios del proyecto

El código fuente, las capturas y la documentación se sincronizaron en los siguientes repositorios públicos:

- GitHub: https://github.com/mattia4567/piedrapapelotijera.git
- GitLab: https://gitlab.com/Masterpuchi23/piedrapapelotijera.git

La estructura de carpetas respeta la organización exigida por la cátedra:

```
piedrapapelotijera/
 |-- .gitattributes
 |-- .gitignore
 |-- LICENSE
 |-- README.md
 |-- docs/
 | |-- InformeEEST1_LPR2026_ACT07_G03_Informe_v1.0.0.pdf
 | |-- CHANGELOG.md
 | `-- manuales/ (manual_programador y manual_usuario, .pdf y .md) |-- src/
 | |-- main.cpp
 | `-- historial_partidas.txt
 `-- capturas/
 |-- jugada_consola.png
 `-- archivo_disco.png
```

Diseño del programa

Modelado de datos con struct

Cada partida se representa con la estructura *Partida*, que reúne en una sola unidad los cinco datos del juego (ver Tabla 1).

Tabla 1

Campos de la estructura Partida

Campo Tipo Descripción

idPartida int Número identificador de la partida

jugador string Nombre o nickname del jugador

eleccionJugador string Piedra, Papel o Tijera elegido por el usuario eleccionIA

string Jugada generada aleatoriamente por la IA resultado string Victoria, Derrota o Empate

Nota. Elaboración propia a partir del código fuente de la actividad.

```
struct Partida {
 int idPartida;
 string jugador;
 string eleccionJugador;
 string eleccionIA;
 string resultado;
};
```

Funciones modulares

El programa se divide en funciones: *main()* muestra el menú principal (jugar, ver historial o salir) dentro de un ciclo *do-while*; *jugarPartida()* pide el nombre y la jugada, genera la elección de la IA con *rand()* inicializada con *srand(time(0))*, determina el resultado y llama a guardar; *guardarEnDisco()* escribe la partida en el archivo; y *leerHistorialDisco()* lee y muestra todas las partidas guardadas.

Reglas del juego

Si ambas elecciones son iguales el resultado es Empate. El jugador gana si elige Piedra contra Tijera, Papel contra Piedra o Tijera contra Papel; en cualquier otro caso el resultado es Derrota.

Persistencia en disco

Escritura con ofstream

La función *guardarEnDisco()* abre el archivo en modo *append* y escribe los datos separados por comas (formato CSV), de modo que cada partida ocupa una línea:

```
ofstream archivo("src/historial_partidas.txt", ios::app);
archivo << p.idPartida << "," << p.jugador << ","
 << p.eleccionJugador << "," << p.eleccionIA << ","
 << p.resultado << endl;
archivo.close();
```

Lectura con ifstream

La función *leerHistorialDisco()* abre el archivo con *ifstream* y lo recorre con *getline()* usando la coma como delimitador, mostrando cada registro y el total de partidas leídas. Si el archivo no existe, informa que aún no hay partidas guardadas.

5

```
ifstream archivo("src/historial_partidas.txt");
while (getline(archivo, idStr, ',') && getline(archivo, jugador, ',') &&  getline(archivo,
elJug, ',') && getline(archivo, elIA, ',') &&  getline(archivo, resultado)) {
 total++;
 cout << "Partida #" << idStr << " | Jugador: " << jugador ...
 }
```

Demostración de persistencia

Para demostrar la persistencia se jugó una partida desde la terminal, se cerró la consola, se volvió a abrir el ejecutable y se eligió la Opción 2 del menú, que mostró el historial guardado. Las evidencias se encuentran en la carpeta *capturas/* del repositorio: *jugada_consola.png* (menú y partida jugada) y *archivo_disco.png* (archivo .txt abierto). **Cuestionario técnico de control**

1. RAM vs. disco

Al cerrar la consola, el programa termina y las variables almacenadas en la memoria RAM se pierden, porque esta memoria es volátil. Sin embargo, cada partida fue escrita en *historial_partidas.txt*, que está en el almacenamiento secundario (disco rígido o de estado sólido), que es no volátil. Al abrir nuevamente el ejecutable y elegir la Opción 2, el programa lee ese archivo con *ifstream* y recupera los datos.

2. Uso de ios::app

Si se usara *ofstream archivo("historial_partidas.txt");* sin *ios::app*, el archivo se abriría en el modo de escritura por defecto (*ios::out*), que borra el contenido existente al abrirlo. Cada partida nueva eliminaría el historial anterior y el archivo solo conservaría la última partida jugada. El modo *ios::app* agrega los datos al final sin borrar lo ya guardado. ***3. Manejo del búfer (cin.ignore())***

Después de leer la opción del menú con *cin >> opcion*, el Enter (\n) que presionó el usuario queda

pendiente en el búfer de entrada. Si se llamara directamente a *getline(cin, p.jugador)*, esta función leería ese salto de línea como una línea vacía y el programa saltearía la pregunta del nombre. *cin.ignore()* descarta ese carácter pendiente y permite que *getline()*

4. Agrupación con struct

La estructura *Partida* reúne variables de distinto tipo en una sola entidad, lo que ofrece varias ventajas frente a pasar cinco variables sueltas: las funciones reciben un solo parámetro (*const Partida& p*) en lugar de cinco, el código es más legible y menos propenso a errores de orden de argumentos, y agregar un campo nuevo no obliga a modificar la firma de cada función. Además, el dato queda representado como un registro lógico, equivalente a una fila del archivo.

5. Aplicación al P.I.A.

Esta misma técnica puede aplicarse al Proyecto Integrador Anual adaptando los campos de la estructura para registrar eventos reales del prototipo, por ejemplo: lecturas de sensores con fecha y hora, registros (logs) de usuarios, estado de stock o fichaje de turnos. También permite guardar configuraciones del hardware en un archivo .txt o .csv que se recupere al reiniciar el sistema, de modo que los datos no se pierdan al apagar el equipo. **Conclusión**

La actividad permitió comprender en la práctica la diferencia entre la memoria RAM y el almacenamiento en disco, y cómo un programa puede conservar información entre ejecuciones. El uso de *struct* facilitó organizar los datos de cada partida, y las clases *ofstream* e *ifstream* junto con el modo *ios::app* permitieron guardar y recuperar el historial sin perder partidas anteriores.

También se reforzaron habilidades de trabajo profesional: compilar desde la terminal, limpiar el búfer de entrada, documentar el proyecto y versionarlo en dos repositorios remotos (GitHub y GitLab). Estos conocimientos son aplicables al Proyecto Integrador Anual para registrar datos reales de un prototipo, y representan un paso importante en el inicio de la programación con archivos.

Referencias

cppreference.com. (s. f.). *std::ios_base::openmode*. https://en.cppreference.com/w/cpp/io/ios_base/openmode

cppreference.com. (s. f.). *std::basic_istream<CharT,Traits>::ignore*. https://en.cppreference.com/w/cpp/io/basic_istream/ignore

cppreference.com. (s. f.). *std::basic_fstream*. https://en.cppreference.com/w/cpp/io/basic_fstream

Git. (s. f.). *git-remote Documentation*. https://git-scm.com/docs/git-remote Mattia, F., Orue, M., Ferrari, U., y Del Pino, L. (2026). *piedrapapelotijera* [Repositorio de código]. GitHub. https://github.com/maximoorue30-tech/piedrapapelotijera.git

Mattia, F., Orue, M., Ferrari, U., y Del Pino, L. (2026). *piedrapapelotijera* [Repositorio de código]. GitLab. https://gitlab.com/maximoorue30-tech/piedrapapelotijera.git Mansilla Muñoz York, E. (2026). *Actividad 7: Mini juego "Piedra, Papel o Tijera" con persistencia de datos (struct + fstream)* [Material didáctico]. E.E.S.T. N.º 1 "Eduardo Ader".